

Priority Queue

- Priority Queue is type of Queue that arranges elements based on their priority values.
 - Elements with higher priority values are retrieved before elements with lower priority.
 - Higher priority elements placed near front pointer of Queue, so that they can be dequeued first.
- Eg. of Priority Queue is Dijkstra algorithm (shortest path algo)

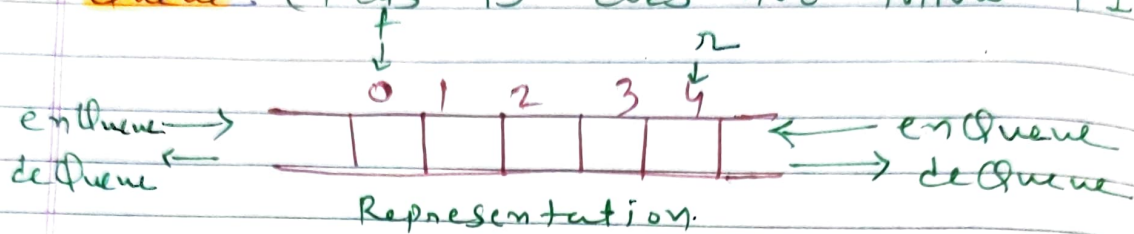
Properties of Priority Queue

Priority Queue is extension of Queue with following property-

1. Every item has priority associated with it.
2. An element with high priority is dequeued before an element with low priority.
3. If two elements have same priority, they are served according to their order in Queue.

What is Deque (double ended Queue)

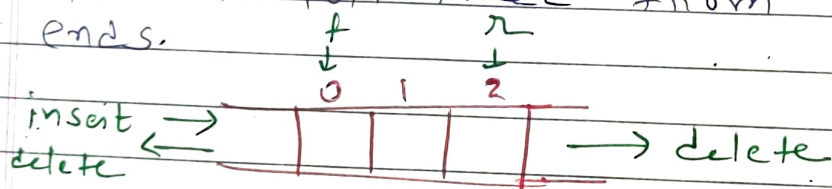
→ Dequeue is linear data structure where the insertion and deletion operations are performed from both ends. We can say that Dequeue is generalized version of Queue. (thys is does not follow FIFO rule)



Types of Dequeue.

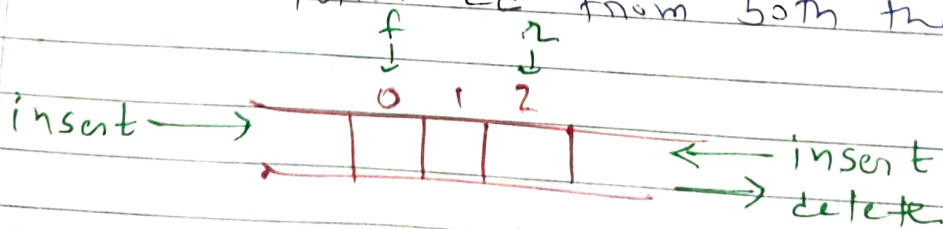
1. Input restricted Queue

Here insertion/enqueue operation performed at only one end But deletion/dequeue operation performed from both the ends.



2. Output restricted Queue

Here deletion/dequeue operation performed at only one end but insertion/enqueue operation performed from both the ends.



→ Dequeue follows property of Both stack (LIFO) and Queue (FIFO)

Stack :- insert, delete from same end
Queue :- insert, delete from different ends.

Operations Performed on deque

1. Insertion at front
2. Insertion at rear (already done in chap-4)
3. Deletion at front (already done in chap-4)
4. Deletion at rear

get front
get rear

We can also perform Peek operation in Deque from that operation we can get front, rear element of Deque because insertion from front & rear both is possible so peek represents first element according to above operations.

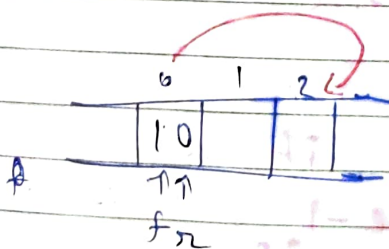
Implementation done using circular array)

⇒ Insertion at front:-

Here element is inserted from front of Queue to perform this, we have to check

1. the Queue is full or not and if not then we have to decrement front and place the value at position of front

eg

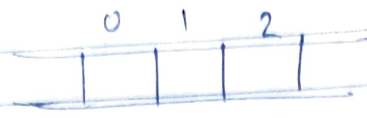


→ if i want to insert 20 from front and front is at pos 0 then

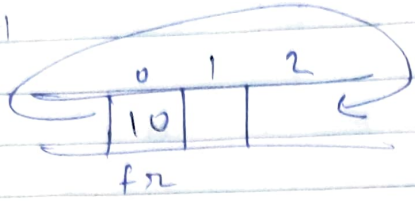
i have to traverse front to size-1 and place the value at $Q[2] = 20$

eg

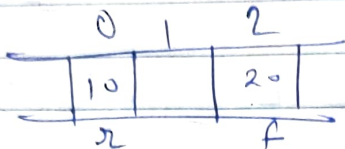
$f = r = -1$



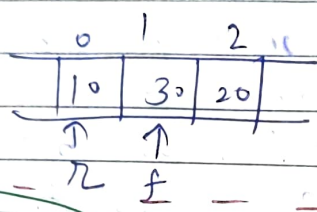
(initially Queue empty)



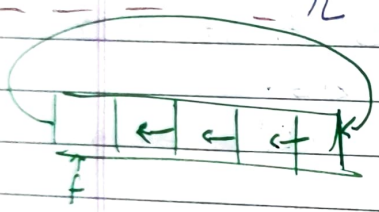
insert 10 at front
set $f = r = 0$ and
 $Q[F] = 10$



insert 20 at front
Set $f = \text{Size} - 1 = 2$



Because we use Circular Queue
at 0, 1, 2
insert 30 at front
(we have to do $f--$)



follow this order ✓

Note: if Queue is empty and want to insert at front then set $\text{front} = \text{rear} = 0$ and place the value otherwise check position of front

if $\text{front} < 1$ (front = 0) else
Set $\text{front} = \text{Size} - 1$ Set $\text{front} = \text{front} - 1$
and place the value

insertion at front: if front is at 0 then set it to $\text{Size} - 1$ else front --
deletion at front: if front is at $\text{Size} - 1$ then set it to 0 else $\text{front}++$

$\leftarrow, \leftarrow, \leftarrow, \leftarrow, \leftarrow$
 $f \rightarrow f, \pi$

0 to S-1 then

f --

Vidyalekhan

DATE
PAGE

Logic / function of Insert at Front

void Insertfront(int y)

{

if (front == 0 && rear == size-1) ||
(front == rear + 1)

{

s.o.p ("Overflow");

}

else

{

if (front == -1)

{

front = 0;
rear = 0;

}

else if (front == 0)

{

front = size-1;

}

else

{

front = front-1;

}

arr[front] = y;

s.o.p (y + "is inserted");

}

}

for adding
1st element
at front

if f=0 &
r != size-1
then for adding
element set
f = S-1

in else case

→ Deletion at Rear

- Here element is deleted from Rear (last position) of Queue
- Before implementing the operation, we have to check for 'underflow' condition
- if $\text{front} = -1$ then underflow else perform $\text{rear}--$ and if $\text{rear} == 0$ then Set $\text{rear} = \text{Size} - 1$

Check for underflow

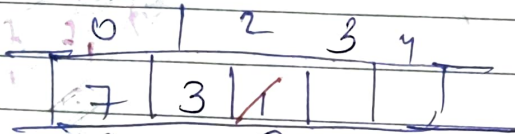
if $\text{rear} == 0$

then $\text{rear} = \text{Size} - 1$

else perform $\text{rear}--$

eg

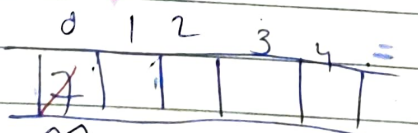
Case-1



↑ ↑
f r



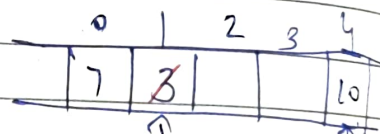
↑ ↑
f r



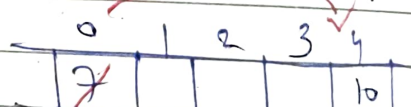
↑↑
fr

Set $\text{f} = \text{r} = -1$

Case-2

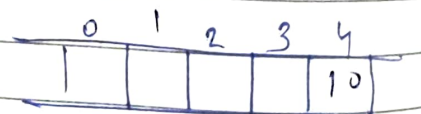


r-- ↑
r



↑ ↑
r f

$\text{r} = \text{Size} - 1$



↑↑
fr

do
 $\text{r}--$
in all
step

Logic for Deletion at Rear

```

Void DeleteRear ( )
{
    if (front == -1 && rear == -1)
    {
        S.o.p ("underflow");
    }
    else
    {
        y = Q[rear];
        S.o.p ("y + " is deleted");
        if (front == rear)
        {
            rear = front = -1;
        }
        else if (rear == 0)
        {
            rear = Size - 1;
        }
        else
        {
            rear--;
        }
    }
}

```

OR

```

if ((f == -1) && (r == -1))
{
    S.o.p ("underflow");
}
else if (f == r)
{
    f = r = -1;
    S.o.p (Q[r] + " is deleted");
}
elseif (r == 0)
{
    S.o.p (Q[r] + " is deleted");
    r = size - 1;
}
else
{
    S.o.p (Q[r] + " is deleted");
    r--;
}

```

1 ele. in queue
200 position
0 element in queue

Other operations possible in DEQUE

1. check Empty :- (checks, that the Queue is Empty or not using $f = -1$ condition)
2. check full :- (checks the Queue is full or not using $f = n+1$ OR $f = 0$ and $r = n$ condition)
3. Get front :- gets front item from Queue
4. Get rear :- gets last item from Queue

Logic for get front

```

Void getfront ()
{
    if ( $f \neq -1$ )
    {
        S.o.p ( $Q[f]$ );
    }
    else
    {
        S.o.p ("front position is at -1");
    }
}

```

Logic for get rear

```

Void getrear ()
{
    if ( $r \neq -1$ )
    {
        S.o.p ( $Q[r]$ );
    }
}

```



```

else
{
    S.o.p ("near is at position -1");
}
}

```

Logic for Displaying elements of DEQUE

```

void display()
{
    int i = f;
    S.o.p ("The Queue is");

    while (i != r)
    {
        S.o.p (A[i]);
        i = (i+1) % size;
    }
    S.o.p (A[r]);
}

```

5. is Empty(): (checks whether Deque is empty or not)
6. is Full(): (checks whether Deque is full or not)

Application of deque

- used as both stack and queue as it supports all operation.
- Navigation of web browser's history
- Palindrome checker (add near, remove near & remove front)